Computing a Maximum Valid Clique Inequality via Row Generation

digital futures

Saga Olivia Tran

KTH Royal Institute of Technology

1. Introduction to Mixed-Integer Programming

Imagine a postal service that wants to plan its delivery routes. It cannot dispatch 28.6 trucks or 52.2 personnel, such counts must be integers. A mixed-integer program (MIP) combines continuous and discrete variables in linear constraints $Ax + By \leq b$, $x \in \mathbb{R}^n$, $y \in \{0,1\}^m$. The binary $y_i \in Y$, $i = 1, \ldots, m$ makes the feasible region a finite set of points rather than a convex set, which makes MIPs hard to solve.

Solvers therefore work with the **LP relaxation**, which replaces $y \in \{0,1\}^m$ with $0 \leq y \leq 1$. The resulting polytope contains every feasible point, as well as fractional points that does not satisfy an integer solution. The resulting *gap* is what make these models expensive. Adding structures that keep all integer points and cuts off fractional ones, can help close this gap. One such structure is a **clique inequality** $\sum_{i \in C} y_i \leq 1$. It states that at most one variable in C can be active ($y_i = 1$, $i \in C$) at a time.

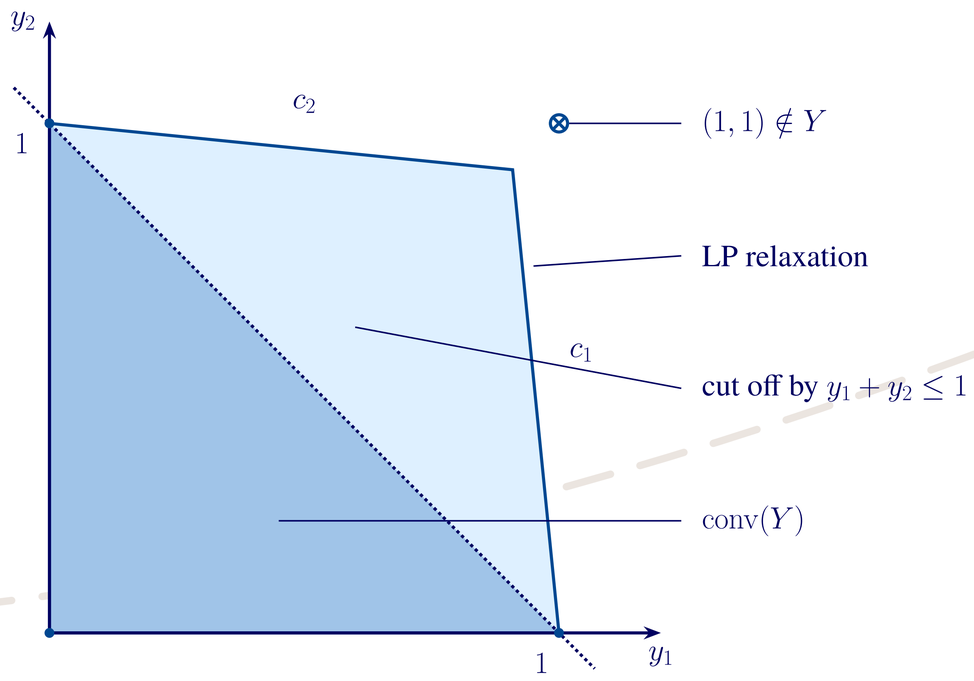


Figure 1. A feasible region defined by two constraints, $c_1 : 10y_1 + y_2 \leq 10$ and $c_2 : y_1 + 10y_2 \leq 10$. Together they exclude $(1,1)$, so $Y = \{(0,0),(1,0),(0,1)\}$, but they meet at $(10/11, 10/11)$ and leave most of the relaxation intact. The clique inequality $y_1 + y_2 \leq 1$ excludes no point of Y yet removes the whole light-blue region, leaving $\mathrm{conv}(Y)$. The larger the cardinality of the set C is, the more of the relaxation is removed, which motivates computing a **maximum valid clique**.

2. Row Generation algorithm

We want to find $C \subseteq \{1, \ldots, m\}$ of maximum size such that $\sum_{i \in C} y_i \leq 1$ holds for every $y \in Y = \{y \in \{0,1\}^m : \exists\, x,\ Ax + By \leq b\}$. Introduce auxiliary variables $z_i \in \{0,1\}, i = 1, \ldots, m$ with $z_i = 1$ if $i \in C$. A clique is valid if $z^\top y \leq 1 \,\forall\, y \in Y$.

Since Y is exponential in size, we impose K constraints at a time. This **master problem** is a relaxation and its optimum $\bar{z}$ is an upper bound of the true maximum clique size. The **separation problem** $\varphi_{MIP}(\bar{z})$ returns the largest number of candidate variables that any feasible solution activates simultaneously. If $\varphi_{MIP}(\bar{z}) \leq 1$, then $\bar{z}$ is a **certified** maximum clique. Otherwise the cut $\bar{y}^\top z \leq 1$ is added to the master problem. Alternating the two problems until $\varphi_{MIP}(\bar{z}) \leq 1$ is the row generation algorithm 1.

Algorithm 1: Row generation for maximum valid clique

Input: A, B, b defining $Ax + By \leq b,\ y \in \{0,1\}^m$

Output: Maximum valid clique $\bar{z}$

$K \leftarrow 0$

repeat

  Solve **master problem**: $\bar{z} \leftarrow \arg\max \sum_i \omega_i z_i$ s.t. $\bar{y}_k^\top z \leq 1,\ k = 1, \ldots, K$

  Solve **separation problem**:

  $\varphi_{MIP}(\bar{z}) \leftarrow \max_{x,y}\{\bar{z}^\top y : Ax + By \leq b,\ y \in \{0,1\}^m\}$, with maximizer $\bar{y}$

  if $\varphi_{MIP}(\bar{z}) > 1$ **then**

    add cut $\bar{y}^\top z \leq 1$ to master problem

    $K \leftarrow K + 1$

  end

until $\varphi_{MIP}(\bar{z}) \leq 1$

return $\bar{z}$

3. Alternative Clique Inequalities

A given MIP can have several different maximal or near-maximal cliques, which exposes different subsets of variables. Each clique reveals a different structural property that can be added back to the original model as a cutting plane.

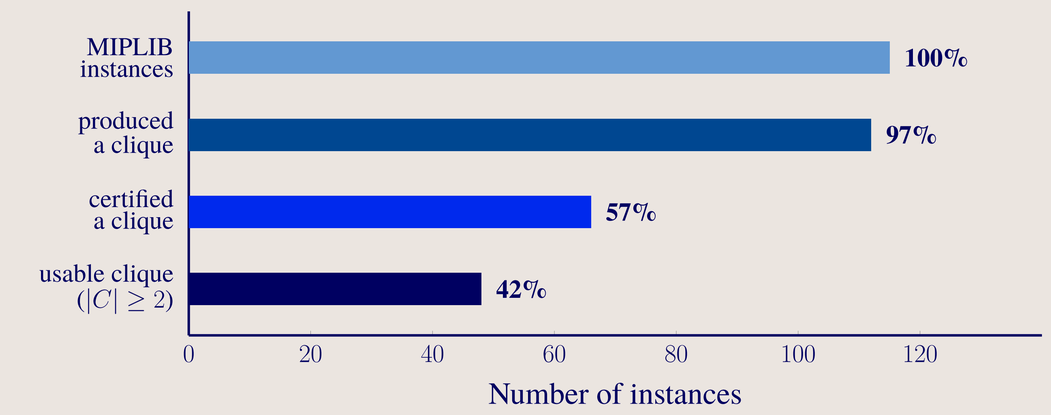
The **alternative clique row generation** calls Algorithm 1 repeatedly on the same instance. The weight factor $\omega_i \in (0,1]$, $i = 1, \ldots, m$ puts a lower weight on $z_i \in C_1$ which pushes the algorithm to choose new variables for C_2 and so on. When a valid clique C_j is found, we add a **cover cut inequality** $\sum_{i \in C_j} z_i \leq \min\{\lfloor \alpha \times |C_j| \rfloor, |C| - 1\}$, to the master problem. The parameter $\alpha \in (0,1]$ is an overlap cap that limits how much of C_j the next alternative clique C_{j+1} may reuse. When deriving alternative cuts, we regulate either ω_i *or* α, that is, $\alpha \in (0,1) \Rightarrow \omega_i = 1 \,\forall\, i = 1, \ldots, m$ and $\omega_i \in (0,1) \,\forall\, i = 1, \ldots, m \Rightarrow \alpha = 1$.

4. Test set-up

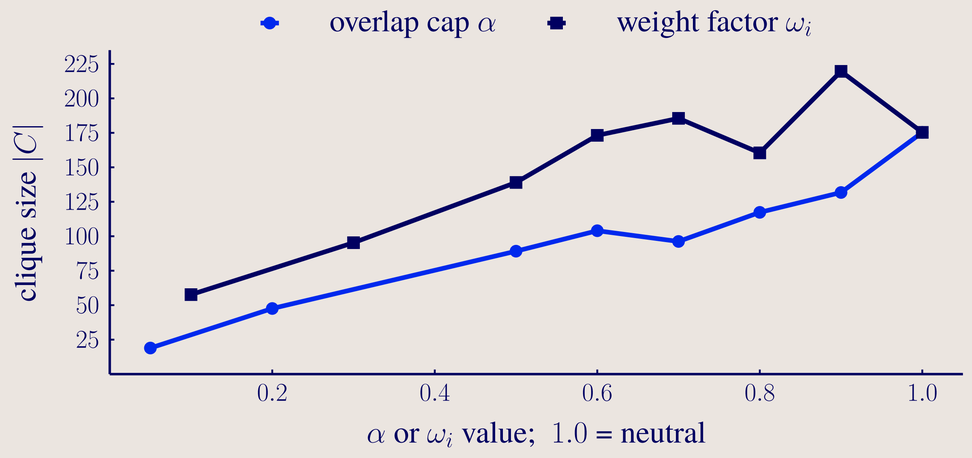
To test the algorithm, we used 115 random instances from the MIPLIB easy-v18 library. Each individual master-separation iteration has a runtime capped at 300s, and the total runtime cap for one instance was set to 1 500s. For each instance, the goal was to derive $n_{alt} = 50$ alternative cliques.

5. Findings

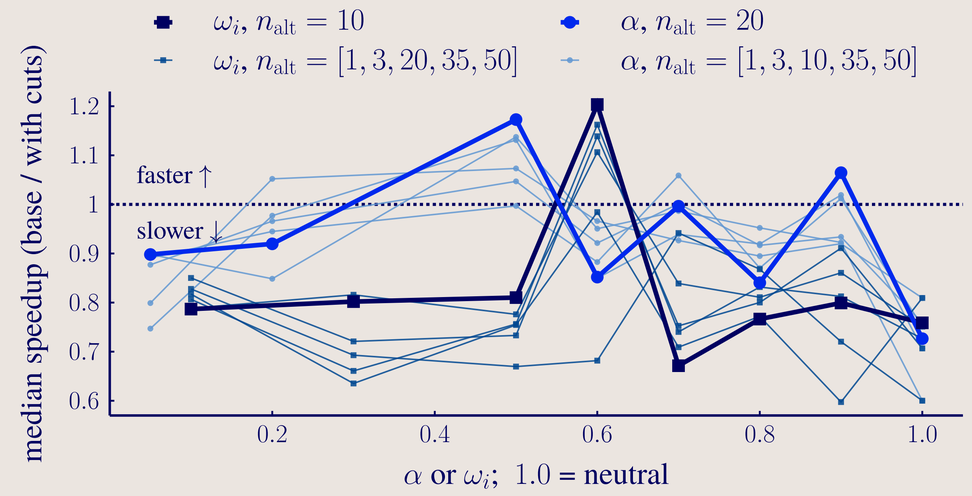
Almost every instance produced at least one clique, but only 42% was certified and of size $|C| \geq 2$. The bottleneck is the separation problem, which is a full MIP re-solve, and on larger instances the runtime budget is exhausted before proving optimality.



Both the overlap cap α and the weight factor ω_i trade clique size against diversity. Tightening either forces C_{j+1} to be nearly disjoint from C_j, which decreases the cardinality $|C|$. Conversely, relaxation towards the neutral value $\alpha = \omega_i = 1$ produces larger cliques, but with substantial pairwise overlap, $|C_i \cap C_j| \approx |C_i \cup C_j|$, for $i \neq j$.



Adding the cliques back as set packing rows shows no systematic increase in Gurobi runtime, when solving the original MIP instances. Gurobi applies presolve, cut generation, and heuristics automatically, so additional constraints change what the solver does rather than only tightening the formulation. Thus, a valid clique that tightens the LP relaxation can still increase the runtime. Even so, combinations of $(\alpha, \omega_i, n_{alt})$ do have a positive effect on the runtime.



6. Conclusion and Future Work

The row generation exposes structurally interesting properties of the instances. Many of the cliques with higher cardinality are not stated in the original formulation, and many do tighten the LP relaxation. That does not necessarily translate to a faster runtime. Currently, we know how much of the relaxation that a clique removes, but not *where* the cut crosses the polytope. It may tighten the feasible region without moving the bound that the solver actually uses. Future work should therefore measure the tightening in the direction of the objective.

QR code

Supervisor: Jan Kronqvist

sagatran@kth.se